AstroGame 2.0 — Phase 1 Handoff

Oct 26, 2026 · @PGCP-AI CDAC Mumbai

Summary

Phase 1 is **complete**. The `SpaceStation` scene can be played from spawn to a mission being accepted, and all 24 checklist tests passed in the Unity Editor on 26 October 2026.

The prototype delivers the first step of the AstroGame mission loop: the player explores Meridian Station in first person, walks to Mission Control, uses the mission console, reads a briefing for mission M01 (Operation First Light, target Kepler-7b) and accepts it. Launching to the planet is a clearly marked placeholder for Phase 2.

Item	Value
Engine	Unity 6.3 LTS (6000.3.27f1), Universal Render Pipeline
Input	Input System package, `InputSystem_Actions` asset
Platform tested	Windows, Unity Editor Play mode
Repository	`github.com/PS2407/AstroGame2` (private), branch `main`
Local path	`C:\Unity Projects\AstroGame2`

Implemented features

All seven Phase 1 objectives are met.

Objective	What was built
Navigable space station	Docking area (8 × 8 m), central corridor (16 × 3 m) and mission-control room (14 × 12 m, 5 m ceiling), generated as editable objects by an Editor script

Objective	What was built
Player controller	First-person walk (3.5 m/s), sprint (6 m/s, Left Shift), mouse look clamped to ±85°, gravity, wall collision via Character Controller
Camera	Child of the player at 1.65 m eye height; body turns left/right, camera tilts up/down
Mission-control area	Mission console, two decorative workstations, observation window with a planet and a 400-star starfield
Interactive objects	Reusable interaction system: camera ray within 2.5 m, `IInteractable` contract, E to interact, crosshair and on-screen prompt
Mission briefing UI	Branded panel with mission title, target planet, description, objectives, resources and constraints; Start Mission, Close and Esc
Expansion path	Mission content stored as a data asset; console and briefing connected through Inspector events, ready for Phase 2 to hook into

Also in place: URP lighting with emissive panels, cursor locking that pauses while the briefing is open, a placeholder **Mission Accepted** confirmation, Git history and a GitHub backup.

Project, folder and scene structure

All project work lives under `Assets/_Project`; the build uses one scene, `SpaceStation`.

```
Assets/
  _Project/
    Art/
      Materials/Station/   11 URP materials (M_Floor, M_Wall, M_GlowCyan,
M_Glass, ...)
      Models/              empty, reserved
      Textures/            empty, reserved
    Audio/                 empty, reserved
    Data/Missions/         Mission_01_FirstLight.asset
    Prefabs/               Environment, Player, Interactables, UI (empty,
```

```
reserved)
    Scenes/                SpaceStation.unity  (build index 0)
    Scripts/
      Editor/              StationBuilder, MissionBriefingBuilder
      Interaction/         IInteractable, PlayerInteractor
      Mission/             MissionConsole, MissionData
      Player/              FirstPersonController
      UI/                  InteractionPromptUI, MissionBriefingUI
    Settings/              empty, reserved
  InputSystem_Actions.inputactions   (Unity template)
  Settings/                URP pipeline assets (Unity template)
  TextMesh Pro/            TMP Essentials
```

The `SpaceStation` scene hierarchy:

```
Directional Light
Global Volume
SpaceStation
  Environment/   DockingArea, Corridor, MissionControl
  Lighting/      9 point lights
  Interactables/ MissionConsole (Desk, StatusStrip, ScreenBezel, Screen)
  Space/         Planet_Kepler, Starfield
  PlayerSpawn
Player           Character Controller, First Person Controller, Player
Interactor
  Main Camera
HUD              Canvas, Interaction Prompt UI
  Crosshair
  InteractionPrompt / PromptLabel
  MissionBriefing  Mission Briefing UI
    Panel / Window / BriefingView, ConfirmationView
EventSystem
```

Scripts and their responsibilities

Nine scripts, each with one job; game code is in the `AstroGame.*` namespaces.

Script	Namespace	Attached to	Responsibility
`FirstPersonController`	AstroGame.Player	Player	Walk, sprint, gravity, mouse look, cursor lock; `SetInputEnabled` pauses control for UI
`IInteractable`	AstroGame.Interaction	(interface)	Contract for usable objects: prompt text, `CanInteract`, `Interact`
`PlayerInteractor`	AstroGame.Interaction	Player	Camera ray within 2.5 m; raises `TargetChanged`; calls `Interact` on E
`MissionConsole`	AstroGame.Mission	MissionConsole	Interactable with prompt text; raises On Activated; locked while in use until `Release`
`MissionData`	AstroGame.Mission	(data asset)	One mission's ID, title, planet, description, objectives, resources and constraints
`InteractionPromptUI`	AstroGame.UI	HUD	Shows or hides "[E] action" from `TargetChanged`; key name read from the input binding
`MissionBriefingUI`	AstroGame.UI	HUD/MissionBriefing	Fills and shows the briefing, pauses the player, Start Mission placeholder, Close and Esc; raises On Opened, On Mission Accepted, On Closed
`StationBuilder`	AstroGame.EditorTools	(Editor menu)	**AstroGame → Build Space Station**; generates the station and its materials
`MissionBriefingBuilder`	AstroGame.EditorTools	(Editor menu)	**AstroGame → Build Mission Briefing UI**; generates the briefing UI and the placeholder mission

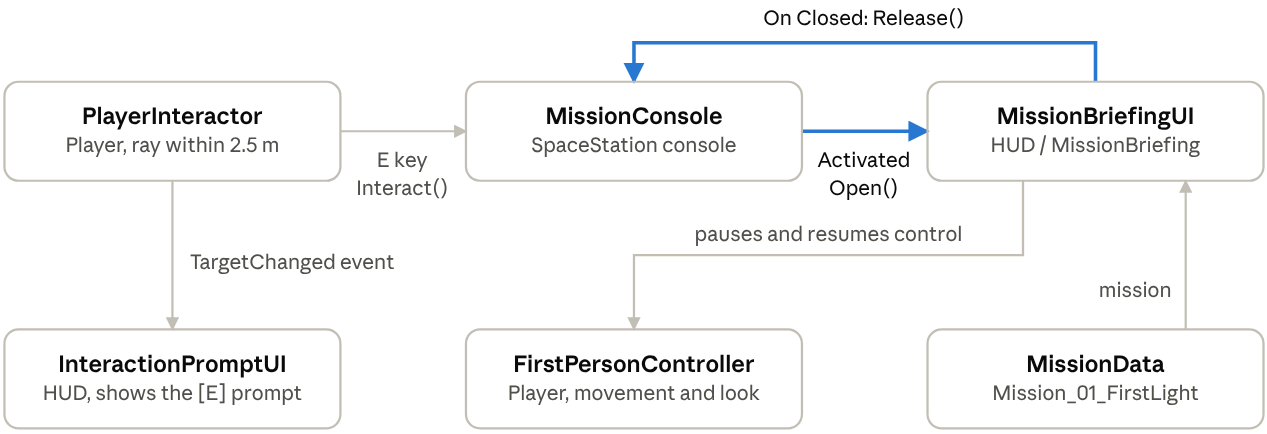
No singletons are used. Keys come from `InputSystem_Actions` (Player/Move, Look, Sprint, Interact; UI/Cancel), never hardcoded.

How the systems connect

Scripts talk through references and events set in the Inspector, so each one stays independent.

Two Inspector events link the console and the briefing in both directions

Highlighted links are UnityEvents set by hand; rerunning a builder menu breaks them



script wiring · 6 components, 2 Inspector events

Pressing E makes `PlayerInteractor` call the console, whose **On Activated** event opens the briefing; the briefing pauses the player and, through **On Closed**, releases the console again. **On Opened** and **On Mission Accepted** are still empty and are the hooks for Phase 2.

Completed tests

All 24 tests passed on 26 October 2026 in Unity Editor Play mode on Windows; the full record is `AstroGame2_Phase1_Test_Checklist.xlsx`.

Group	Tests	Covers	Result
Scene and spawn	3	Scene loads with no errors, correct spawn, a second Play run behaves the same	3 / 3 passed
Movement and camera	5	Walk, sprint, mouse look and clamp, gravity, cursor lock	5 / 5 passed
Collision	2	Walls, crates, desk, workstations and window are solid	2 / 2 passed

Group	Tests	Covers	Result
Interaction	4	Console detected, prompt hides when looking away or out of reach, no prompt elsewhere	4 / 4 passed
Mission briefing	9	Opens on E, player paused, no repeat activation, Close, Esc, console reusable, Start Mission, Acknowledge, accepted state	9 / 9 passed
Stability	1	No recurring Console errors over about 2 minutes	1 / 1 passed

Not yet tested: a standalone Windows build (.exe). All testing so far was inside the Editor.

Known limitations

The most important limitation is that the two builder menus overwrite hand-made changes if run again.

Limitation	Impact	Suggested fix
Build Space Station recreates `SpaceStation` from scratch	Removes the Mission Console script, its On Activated wiring and the Desk's Ignore Raycast layer	Don't rerun it; in Phase 2 save the console as a prefab or retire the builder
Build Mission Briefing UI recreates `MissionBriefing`	Its Player Controller and On Closed connections must be redone, and the console's On Activated link breaks	Save HUD as a prefab once the UI is stable
Mission Accepted state lives in memory only	Resets every time Play starts; nothing is saved	Add a progress/save system in Phase 2
One mission, assigned directly on the briefing	The console can't choose between missions	Give the console its own Mission Data reference and pass it to `Open`
Esc in the Editor also releases the cursor	After closing with Esc, a click on the Game view is needed	Editor-only; check in a standalone build

Limitation	Impact	Suggested fix
No standalone build tested	Unknown behaviour outside the Editor	Make and test a Windows build at the start of Phase 2
Doorways are open frames	No doors open or close yet	Add a door that uses `IInteractable`
No audio, pause menu or settings	Silent prototype; sensitivity editable only in the Inspector	Phase 2 or later
Placeholder visuals	Primitive geometry, a sphere planet, cube stars, no baked lighting	Art pass in a later phase
Template leftovers	`Assets/Scenes/SampleScene`, `TutorialInfo` and `Readme` remain	Delete them in a clean-up commit
Prefab folders are empty	Player and HUD exist only inside the scene	Create prefabs before adding a planet scene

Recommended next milestone (Phase 2)

Recommended Phase 2 goal: one complete mission round trip. The player accepts M01, travels to Kepler-7b, completes one puzzle under a resource limit, returns to the station and reports completion at the console. This finishes the travel, solve and report steps of the original AstroGame loop; the assessment and level unlocking follow in Phase 3.

Suggested order:

1. Housekeeping: make Player and HUD prefabs, save the console as a prefab, remove template leftovers, and test a standalone Windows build.
2. Launch: connect **On Mission Accepted** to a short launch sequence and load a new `Planet_Kepler7b` scene.
3. Planet environment: a small landing site built the same way as the station, reusing the player, HUD and interaction system.
4. First puzzle: the power-routing grid from the M01 briefing (restore three relay beacons without overloading a node), using the rover-battery limit as the resource constraint.
5. Return and report: an interactable on the planet returns the player to the station; the console then offers a **Report Mission** action.

6. Mission state: a small progress object that remembers accepted and completed missions across scenes.

Still out of scope for Phase 2: the assessment engine, adaptive difficulty, the FastAPI backend, PostgreSQL and analytics. Recording simple puzzle events (time taken, moves, resource used) in Phase 2 would give Phase 3's assessment real data to work with.

Phase 2 starts only after you confirm this handoff.